

Project Update — v1

Curiosity-Driven Learning for Emergent Tool Use
Unitree G1 Humanoid Robot · Purdue

Prepared by Yagnesh Dawankar · June 15, 2026

1. The Big Picture (Plain English)

We are teaching a humanoid robot (the Unitree G1) to **figure out how to use tools on its own** — for example, using a stick to reach a far-away object, or standing on a box to press a high button. Instead of telling the robot exactly what to do, we reward it for being **curious**: it gets a bonus for trying things it hasn't seen before. Over millions of practice attempts in simulation, this curiosity pushes it to discover clever solutions by itself.

The ultimate goal is **sim-to-real transfer**: train entirely in a fast physics simulator, then run the learned behavior on the real robot. The project runs over 16 weeks and covers 5 increasingly hard tasks.

Where we are now: Phase 2 (training) is underway. We have built all 5 task environments, trained working policies on Tasks 1 and 4, compressed them into small deployable networks (distillation), and run a deep investigation into why one task does not yet transfer between two different simulators. That investigation is the most important finding so far.

2. What We Researched — The Three Curiosity Methods

"Curiosity" is a bonus reward that encourages exploration. The core idea: the robot keeps a small neural network that tries to *predict* something about each state it visits. When it visits a **new** situation, the prediction is bad — and that prediction error becomes a reward. So the robot is naturally drawn toward the unfamiliar. We compared three ways of doing this:

RND — Random Network Distillation

- A fixed, random "target" network and a second "predictor" network that learns to copy it.
- Reward = how badly the predictor copies the target on the current state. New states = high error = high reward.
- The classic, simplest baseline.

DRND — Distributional RND

- Uses an **ensemble** of several independent (target, predictor) pairs instead of one.
- Reward = the average prediction error across the whole ensemble.
- More stable and informative signal because several predictors must all agree a state is familiar.

RDD — Random Disagreement Distillation

- One fixed target but **several predictors**; reward = how much the predictors *disagree* with each other.
- High disagreement means the state is novel. A bandwidth parameter (σ) controls sensitivity.
- In theory elegant, but in our tests the disagreement signal faded too quickly (more below).

All three share the same plumbing in our code: a single factory `make_curiosity(cfg, obs_dim, device)` builds whichever method we ask for, and the PPO trainer talks to all of them through one common interface. This made head-to-head comparison clean and fair.

3. The Architecture & Models

We use **PPO** (Proximal Policy Optimization), a standard and reliable reinforcement-learning algorithm, as the engine. The curiosity bonus is added on top of the task reward. The brain of the robot is an **Actor-Critic** network with an important twist:

Asymmetric Actor-Critic

- **Actor (the policy that acts):** sees only what a real robot could sense — its own joints and body (proprioception), plus, in the newest version, the position of nearby objects. This keeps it deployable on real hardware.
- **Critic (the value estimator, used only in training):** is allowed to "cheat" and see privileged simulator-only information (exact object states, etc.). This makes training more stable. The critic is thrown away at deployment.
- We actually run **two critics** — one for the external task reward, one for the internal curiosity reward — so the two signals don't get tangled.
- The output is a 43-dimensional Gaussian — i.e. the robot controls 43 joint targets each step.

Component	Inputs (observation size)	Notes
Actor (policy)	109 dims (original) -> 115 (closed-loop)	Proprioception; +object pose in new version
Critic (value)	132 dims (original) -> 138 (closed-loop)	Privileged sim-only info; training only
Action output	43-dim Gaussian	Joint position targets

Key file: `src/utils/networks.py` defines the ActorCritic. The curiosity modules live in `src/curiosity/` (`rnd.py`, `drnd.py`, `rdd.py`), and the PPO training loop is in `src/agents/ppo.py`.

How it runs

Training spins up many copies of the robot in parallel inside **NVIDIA Isaac Lab / Isaac Sim 4.5** (a GPU physics simulator). On the big cluster we use 4096 robots at once for 200M steps; locally on an RTX 3050 (8 GB) we use a smaller 32-robot, 10M-step config to develop and debug.

4. The Five Tasks We Built

Each task is a self-contained simulated scene with its own objects, success condition, and randomization. All five are implemented and pass their unit tests.

#	Task	What the robot must do	Success condition	Status
1	Distant Target	Use a 90 cm stick to reach a far object	Target moved ≥ 0.5 m	Trained
2	Elevated Button	Stand on a 20 kg box to reach a button at 1.8m	Button pressed	Built
3	Occluded Retrieval	Push a stick through a slot to fetch a hidden target	Target moved ≥ 0.5 m	Built
4	Weight Lever	Use a plank + fulcrum to lever up a heavy box	Heavy box lifted ≥ 0.10 m	Trained
5	Composite	Go through a gap, move a box, climb it, press button	Button contact ≥ 2 N	Built

All tasks include **domain randomization** — we randomly vary friction, mass, and other physics properties every reset so the policy doesn't overfit to one exact setting (this is what helps real-world transfer). Tasks

2, 3, and 5 are built and unit-tested but await cluster compute to train at full scale.

Engineering fixes along the way: PhysX requires CPU tensors and all-environment indices to set masses/friction; and the Isaac Lab URDF importer was missing a function in version 4.5, which we guarded so the robot model loads correctly.

5. Training Results So Far

Task 1 — Comparing the three curiosity methods (10M steps each)

Method	Final success	What happened
RND	~0% (likely)	Policy never settled; no useful learning signal recorded
DRND	100%	Sudden 'phase transition' at ~4.8M steps: 0% -> 44% -> 100%
RDD	0%	Episodes collapsed to ~5 steps; disagreement signal decayed too fast

Headline result: DRND is the clear winner. It reached 100% success on Task 1, while RND and RDD failed. The DRND robot learned to solve the task in about 12 steps with a solid positive reward. This validates DRND as our default curiosity method going forward.

Task 4 (Weight Lever) was also trained successfully with DRND, reaching high success rates and producing a usable teacher policy.

6. Distillation (Phase 3) — Shrinking the Policy for Deployment

The training-time policy is big and relies on the privileged critic. For deployment we use **Dagger** (Dataset Aggregation), a method where a small **student** network watches the **teacher** and learns to imitate it, repeatedly correcting its own mistakes over 20 rounds.

- **Task 1 student:** hit 100% success by round 3 and held it through round 20. Training loss dropped from 332,000 down to 75. ~15 minutes total.
- **Task 4 student:** 100% success from round 1; loss dropped to under 1 (an even cleaner fit than Task 1). So inside the simulator, distillation works beautifully — we can compress a winning teacher into a small, real-robot-friendly student without losing performance.

7. The Hard Problem We Are Investigating: Sim-to-Sim Transfer

Before risking the real robot, we test whether a policy trained in Isaac (PhysX physics) also works in a **different** simulator, MuJoCo. If a policy can't even survive a change of simulator, it certainly won't survive the real world. This is a cheap, honest stress test. **Task 1's policy currently fails it — it scores 0% in MuJoCo despite ~77-100% in Isaac.** We spent significant effort finding out exactly why.

What we built to investigate

- A **contract dumper** that records Isaac's exact joint ordering, limits, and default pose (Isaac interleaves left/right joints by kinematic depth — a subtle ordering bug source we caught).
- A **ground-truth rollout recorder** in Isaac (confirmed the student gets 9/10 there).
- A full **MuJoCo evaluation harness** that rebuilds the same observation, action mapping, and tools.

Bugs we found and fixed by careful comparison

- **Foot-contact mismatch:** MuJoCo only reported contact on one ankle part; fixed so it matches Isaac **exactly** (observation difference 0.0).
- **Leg collapse:** the robot's legs sagged under MuJoCo's torque model; we stiffened the leg gains 20x to match how Isaac's actuators behave, so the robot stands at the right height.

- **Friction matched** on stick and target.

After fixing all the harness bugs, the observation and action contract is **verified identical** to Isaac (all 43 actions agree). And yet transfer is **still 0%**. That tells us the problem is **not a coding bug** — it is something deeper about the policy itself.

8. Root Cause — Why Task 1 Doesn't Transfer (Confirmed)

After step-by-step tracing in both simulators, we have a confident diagnosis:

- **The winning Task 1 behavior is a ballistic lunge, not a careful reach.** Success in Isaac comes from *propelling the stick* at the target in a fast, semi-ballistic motion — the hands actually stay over a meter away.
- **The policy is 'bang-bang'.** Every one of the 43 joint commands is slammed to its maximum (± 1700). It's an all-or-nothing motion finely tuned to PhysX's exact physics.
- **Such a motion is maximally sensitive to the physics engine.** The identical command sequence in MuJoCo produces a totally different trajectory: the robot launches off the ground (both feet leave within 2 steps) and the stick gets knocked sideways, away from the target.
- **The policy is blind and open-loop.** It only senses its own body, so once the grasp/launch goes wrong it cannot see or correct the mistake.

Bottom line: This is genuine **over-fitting to PhysX physics**, not a harness bug. A precise open-loop lunge that works in one simulator simply won't reproduce in another.

Fixes we already tried (and what they taught us)

Attempt	Result	Lesson
Tanh + saturation penalty (smoother actions)	Isaac still 100%, MuJoCo still 0%	Smoothing is necessary but not sufficient
Closed-loop obs (add object positions) + restitution	Isaac 77%, MuJoCo 15% Isaac 70%, MuJoCo 15% Isaac 70%, MuJoCo 15% Isaac 70%, MuJoCo 15%	Even seeing objects didn't fix a ballistic motion

Importantly, smoothing the actions did **not** hurt Isaac success — so we can make the policy gentler for free. The remaining issue is the *nature of the motion* (a lunge), not its smoothness.

9. Next Steps

To make Task 1 transfer (teacher-side fix)

- Add **action-rate and torque penalties directly in PPO** so the teacher learns a smooth, quasi-static reach instead of a violent lunge from the start.
- Apply **heavier domain randomization** during training: PD gains, restitution, contact stiffness, and control latency — so the policy can't rely on one exact physics setting.
- One ~4-hour retrain, then re-distill and re-evaluate in MuJoCo.
- **Alternative demonstration:** Task 4 (the lever) is naturally slow and quasi-static, not ballistic, so it is a much better candidate to show successful transfer first.

Project-level

- **Critical blocker:** RCAC cluster allocation. Full-scale training of Tasks 2, 3, and 5 — and all of Phase 4 (sim-to-real) — depend on getting compute on the cluster.
- Once a transferable policy exists, extend the MuJoCo cross-validation to Task 4.

10. One-Paragraph Summary for Leadership

We have a working research pipeline end-to-end: five tool-use tasks built and tested, three curiosity methods compared head-to-head (**DRND clearly wins, reaching 100% on Task 1**), and a distillation step that cleanly compresses winning policies for deployment. Our most valuable finding is a **rigorous, well-diagnosed understanding of why one policy doesn't yet transfer across simulators**: the learned motion is a physics-specific ballistic lunge from a blind, open-loop policy. We have verified this is a real learning/physics issue, not a software bug, and we have a concrete plan to fix it (smoother training rewards + stronger domain randomization). The main external dependency is cluster compute to scale the remaining tasks.

End of report · project_updatev1